



Block 4 – Cloud Computing

S2: Cloud Computing & Cloud Architectures

- 1 From Virtualization to Cloud
- 2 Fundamentals of Cloud Computing
- 3 Cloud & Container Platforms
- 4 Cloud Network Architecture
- 5 Cloud Network Services
- 6 The Cloud Trade-Off
- 7 Session Summary

From Virtualization to Cloud

- 1 From Virtualization to Cloud
- 2 Fundamentals of Cloud Computing
- 3 Cloud & Container Platforms
- 4 Cloud Network Architecture
- 5 Cloud Network Services
- 6 The Cloud Trade-Off

If We Can Virtualize Everything... Why Not Sell It?

*Virtualization is the technology.
Cloud computing is the business model.*

Learning objectives:

1. Understand cloud computing's definition and essential characteristics
2. Distinguish IaaS (Infrastructure as a Service), PaaS (Platform as a Service), and SaaS (Software as a Service)
3. Design a cloud virtual network with subnets, security controls, and network services

Meet the Scenario: A Startup from Zero

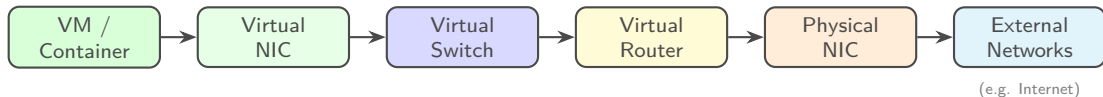
Running example for this session

CloudBite is a 3-person startup that just received seed funding. They need to deploy a food-delivery web app for thousands of users **by next month**. They have laptops, code, and zero infrastructure.

Every section answers the next question CloudBite faces:

1. “We know about VMs. Why not just buy servers?” → **From Virtualization to Cloud**
2. “What exactly is cloud computing?” → **Fundamentals**
3. “Which platform do we use?” → **Platforms**
4. “We picked AWS. Now we need a network.” → **Cloud Network Architecture**
5. “We need HTTPS, traffic distribution, and office access.” → **Network Services**
6. “Is the cloud always the right choice?” → **The Cloud Trade-Off**

Recap: What We Virtualized (Session 1)



- **Compute:** VMs share physical hardware via hypervisors; containers share the host kernel directly
- **Networking:** Virtual NICs, Switches, and Routers replicate the physical stack in software
- **Isolation:** VLANs and overlay networks separate tenants on shared infrastructure

The Leap to Cloud

- Virtualization = technology (**how** you run multiple workloads)
- Cloud = business model (**what** you sell to customers)

Virtualization	Cloud Computing
Run multiple VMs/containers on one host	Offer compute as a service
Internal IT optimization	External/internal consumption
Manual provisioning possible	Automated, self-service
You own the hardware	Provider owns the hardware

Key difference: **automation and self-service** (no phone calls, no tickets, no waiting).

Discussion: From Virtualization to Cloud

Your company already runs VMs on its own servers. Why would you move to the cloud instead of keeping everything in-house?

Think about:

- Capital expenditure vs operational expenditure
- How fast can you scale today?
- Who patches the hardware at 3 am?
- What happens when a disk fails?

Fundamentals of Cloud Computing

- 1 From Virtualization to Cloud
- 2 Fundamentals of Cloud Computing**
- 3 Cloud & Container Platforms
- 4 Cloud Network Architecture
- 5 Cloud Network Services
- 6 The Cloud Trade-Off

CloudBite Asks: “What Exactly Is Cloud Computing?”

- The founders are convinced: buying servers is not an option
- But “the cloud” is a vague term; every vendor defines it differently
- Before choosing a provider, CloudBite needs to understand **what cloud computing actually means** and **what service model fits their needs**

Let us start with the industry-standard definition.

Cloud Computing: Definition (NIST)

NIST SP 800-145 (2011)

*"A model for enabling ubiquitous, convenient, **on-demand network access** to a **shared pool** of configurable computing resources that can be rapidly provisioned and released with **minimal management effort**."*

Three phrases to remember:

- **On-demand**: you get resources immediately, no human approval needed
- **Shared pool**: underlying hardware is shared across many customers (multi-tenancy)
- **Minimal effort**: you click a button (or call an API); the provider handles the rest

1

¹NIST SP 800-145, Mell & Grance, 2011. Still the standard definition used industry-wide.

Five Essential Characteristics

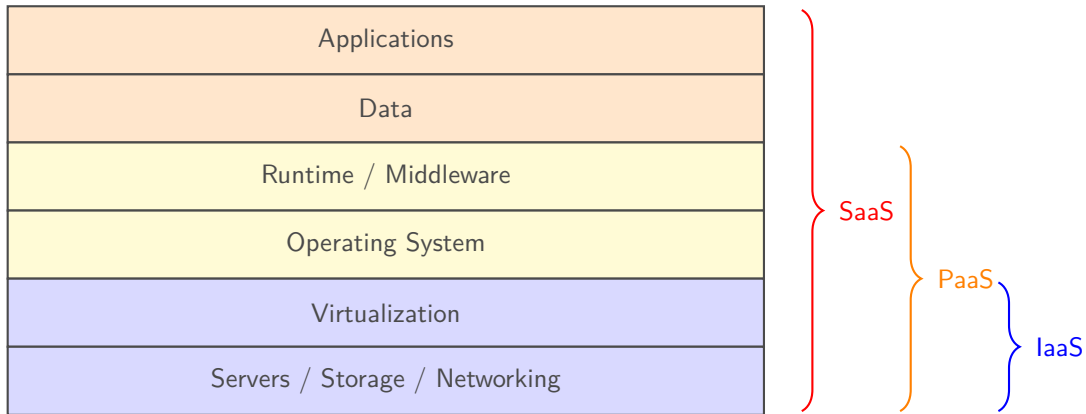
Characteristic	What it means
On-demand self-service	Provision via portal or Application Programming Interface (API), no human intervention
Broad network access	Available from any device over the network
Resource pooling	Shared infrastructure, multi-tenant
Rapid elasticity	Scale up/down automatically
Measured service	Pay only for what you use

- **All five** must be present to qualify as “cloud computing”

2

²NIST SP 800-145, Mell & Grance, 2011.

Cloud Service Models: The Stack



IaaS, PaaS, SaaS: In Practice

IaaS, PaaS, SaaS: In Practice

IaaS (e.g. AWS EC2, Azure VMs, Google Compute Engine):

- Provider gives you VMs, storage, networks. You manage everything else
- “I want a Linux server in Frankfurt, 4 CPUs, 16 GB RAM, right now”

IaaS, PaaS, SaaS: In Practice

IaaS (e.g. AWS EC2, Azure VMs, Google Compute Engine):

- Provider gives you VMs, storage, networks. You manage everything else
- “I want a Linux server in Frankfurt, 4 CPUs, 16 GB RAM, right now”

PaaS (e.g. Google App Engine, AWS Elastic Beanstalk, Heroku):

- Provider gives you runtime + tools. You only manage your code and data
- “I wrote a Python web app. Just run it, I do not care about servers”

IaaS, PaaS, SaaS: In Practice

IaaS (e.g. AWS EC2, Azure VMs, Google Compute Engine):

- Provider gives you VMs, storage, networks. You manage everything else
- “I want a Linux server in Frankfurt, 4 CPUs, 16 GB RAM, right now”

PaaS (e.g. Google App Engine, AWS Elastic Beanstalk, Heroku):

- Provider gives you runtime + tools. You only manage your code and data
- “I wrote a Python web app. Just run it, I do not care about servers”

SaaS (e.g. Google Workspace, Microsoft 365, Slack, Zoom):

- Provider gives you a complete application. You manage your data only
- “I need email for 500 employees. I do not want to run a mail server”

Service Models: Who Manages What?

Component	IaaS	PaaS	SaaS
Applications	You	You	Provider
Data	You	You	You
Runtime / Middleware	You	Provider	Provider
Operating System	You	Provider	Provider
Virtualization	Provider	Provider	Provider
Hardware	Provider	Provider	Provider

- IaaS → SaaS: **less control, less responsibility**
- Major providers (AWS, Azure, GCP) offer all three models

Discussion: Cloud Fundamentals

Case A: University

A large university wants a web tool for students to submit assignments. The IT department has 10 staff. Student data must stay within EU jurisdiction. Budget is fixed yearly.

Which model? Why? - Control over data location needed - Existing IT team can manage servers - Ready-made products exist (Canvas, Moodle) - No need to scale suddenly

Case B: CloudBite

CloudBite (3 engineers) needs to run a web app, store orders, and send email notifications. They must ship in 4 weeks and cannot afford to manage servers.

Which model? Why? - No ops bandwidth: avoid IaaS if possible - Web app: PaaS (just deploy the code) - Email: SaaS (Mailchimp, SendGrid) - Database: PaaS managed DB, or IaaS if they need fine-grained control

Cloud & Container Platforms

- 1 From Virtualization to Cloud
- 2 Fundamentals of Cloud Computing
- 3 Cloud & Container Platforms**
- 4 Cloud Network Architecture
- 5 Cloud Network Services
- 6 The Cloud Trade-Off

CloudBite Asks: “Which Platform Do We Use?”

- CloudBite decided on **IaaS**: they want full control over the stack (they are engineers, after all)
- Now they must choose: run their own infrastructure (private cloud) or rent from a provider (public cloud)?
- They also hear about **Kubernetes** everywhere; should they use containers?

Let us look at the platforms available.

Putting It All Together

Building blocks (Session 1)

- Virtual NIC
- Virtual Switch (bridge)
- Virtual Router
- VLAN / overlay (VXLAN)
- Hypervisor / container runtime

These components are configured and wired together by platforms.

Platforms that manage them

- **Cloud platforms:** orchestrate VMs, storage, networking at scale (IaaS)
 - OpenStack, VMware vSphere, AWS, Azure, GCP
- **Container orchestration:** deploy and scale containerized apps
 - Docker, Kubernetes (K8s), OpenShift

Private Cloud Platforms

- **Self-managed:** you own or rent the servers, install and manage the virtualization software yourself
- Full control over hardware and data; requires dedicated staff and expertise
- Typical use: regulated industries (banking, healthcare), large enterprises with existing data centers

Platform	Description	Logo
OpenStack	Open-source, widely used in private clouds	 openstack®
VMware vSphere	Enterprise leader, proprietary	
Proxmox VE	Open-source, gaining traction since recent VMware price rises	

Public Cloud Platforms

- **Managed / Public:** rent capacity on demand, pay-as-you-go
- Provider handles hardware, networking, cooling, security, and updates
- Maps directly to the IaaS model: provider manages compute, storage, and networking

Provider	Description	Logo
Amazon Web Services (AWS)	Market leader since 2006	
Microsoft Azure	Strong enterprise integration	
Google Cloud Platform (GCP)	Data and AI focus	

Container Orchestration Platforms

- **Self-hosted / Private:** you install and operate on your own servers

Platform	Description	
Docker	Container runtime; works well on a single host Managing hundreds of containers across hosts? Unmanageable	
Kubernetes (K8s)	Created to solve containers running at scale : scheduling, scaling, networking, self-healing	
OpenShift	Enterprise K8s by Red Hat; adds security, CI/CD, UI	

- **Hosted / Public:** cloud providers offer the same platforms as managed services. Each provider uses different names for essentially the same product (e.g., AWS ECS/EKS, Azure AKS, Google GKE, OpenShift on all major clouds).

Discussion: Platforms

A startup with 5 engineers needs to deploy a web application that may go from 100 to 100,000 users. Would you build a private cloud or use a public one? Why?

Think about:

- Upfront cost and time to market
- Who manages infrastructure at night?
- Can 5 engineers run a data center?
- What if traffic spikes overnight?

Cloud Network Architecture

- 1 From Virtualization to Cloud
- 2 Fundamentals of Cloud Computing
- 3 Cloud & Container Platforms
- 4 Cloud Network Architecture**
- 5 Cloud Network Services
- 6 The Cloud Trade-Off

CloudBite Asks: “We Picked AWS. Now What?”

- CloudBite chose **AWS** (public cloud, IaaS): no upfront cost, instant global reach
- First task: create an **isolated network** for their application
- They need a **public subnet** for the web frontend and a **private subnet** for the database
- How do subnets, route tables, gateways, and firewalls work in the cloud?

Time to build CloudBite's virtual network from scratch.

Virtual Networks in the Cloud

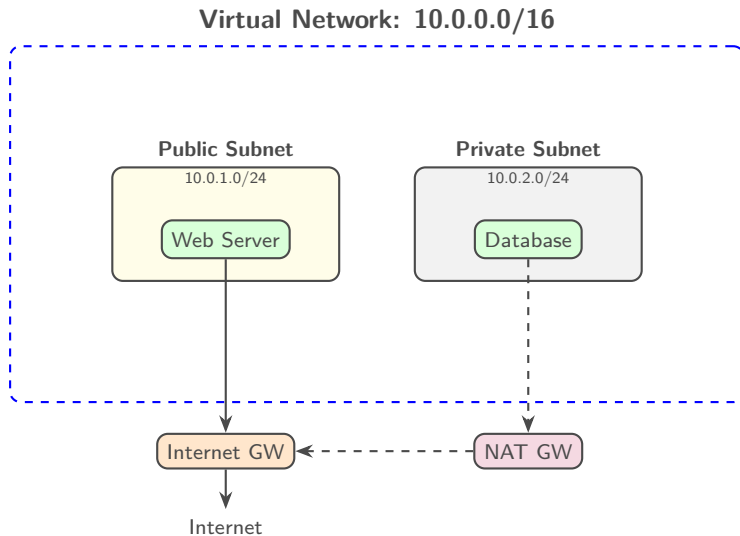
- Every cloud provider lets you create your own **isolated virtual network**
 - AWS: Virtual Private Cloud (VPC); Azure: Virtual Network (VNet); GCP: VPC Network
- Think of it as your **private data center network** in the cloud
- You define:
 - **Address space** (e.g., 10.0.0.0/16)
 - **Subnets** (subdivisions of the address space)
 - **Routing rules** and **security policies**

Key: under the hood, cloud providers use **overlay networks** (covered in Block 5) to isolate tenants at scale.

3

³AWS, "Amazon VPC User Guide"; Azure, "Virtual Network Documentation"; GCP, "VPC Documentation."

Virtual Network Architecture



Subnets: Public vs Private

Public subnet:

- Has a route to an **Internet Gateway**
- Instances can have **public IP addresses**
- Accessible from the Internet (if security rules allow)
- Use for: web servers, load balancers, bastion hosts

Private subnet:

- **No direct route** to the Internet
- Instances only have **private IPs**
- Can access Internet outbound via **NAT Gateway**
- Use for: databases, internal services, application backends

Route Tables

- Each subnet is associated with a **route table**
- Route table = set of rules determining where traffic goes

Destination	Target	Subnet type
10.0.0.0/16	local (within virtual network)	Both
0.0.0.0/0	Internet Gateway	Public
0.0.0.0/0	NAT Gateway	Private

- **Local route:** traffic within the virtual network stays inside (always present)
- **Default route** (0.0.0.0/0): where to send everything else
- Public subnets → IGW; Private subnets → NAT GW

Internet Gateway vs NAT Gateway

Internet Gateway (the front door):

- **Bidirectional**: the world can reach your instance, and it can reach the world
- Instance needs a **public IP**
- Use for: web servers, load balancers, anything public-facing

NAT Gateway (the fire exit):

- **One-way**: your instance can reach the Internet (e.g., download updates, call APIs), but nobody outside can reach it directly
- Instance keeps a **private IP** only
- Use for: databases, backends that need to download patches but must stay hidden

Same **NAT concept** from **Block 3**, now as a managed cloud service.

Security Groups: Instance-Level Firewall

- **Security group** = virtual firewall attached to an instance
- Rules specify allowed **inbound** and **outbound** traffic
- **Stateful**: allow inbound → response automatically allowed

Direction	Protocol	Port	Source/Dest
Inbound	TCP	80	0.0.0.0/0
Inbound	TCP	443	0.0.0.0/0
Inbound	TCP	22	10.0.0.0/16
Outbound	All	All	0.0.0.0/0

4

⁴AWS, "Security Groups for Your VPC," docs.aws.amazon.com.

Network ACLs (Access Control Lists): Subnet-Level Firewall

- **Network ACL** = firewall at the **subnet** level
- Rules evaluated in **order** (numbered, first match wins)
- **Stateless**: must explicitly allow both inbound AND outbound

Rule #	Direction	Protocol	Port	Action
100	Inbound	TCP	80	ALLOW
200	Inbound	TCP	443	ALLOW
*	Inbound	All	All	DENY

5

⁵AWS, "Network ACLs," docs.aws.amazon.com.

Security Groups vs Network ACLs

	Security Groups	Network ACLs
Scope	Instance level	Subnet level
State	Stateful	Stateless
Rules	Allow only	Allow + Deny
Evaluation	All rules evaluated	First match wins
Default	Deny all inbound	Allow all

Defense in depth: use both together.

- Security Groups → fine-grained per-instance control
- Network ACLs → broad subnet-level guardrails

Common mistake

ACLs are **stateless**: if you allow inbound TCP port 80, you must *also* explicitly allow outbound ephemeral ports (1024–65535) for the response. Forgetting this blocks return traffic.

Discussion: Cloud Networking

You deploy a web application in your cloud virtual network. The web server needs to be public, but the database must not be reachable from the Internet. How?

Think about:

- Public subnet for web server (needs Internet Gateway)
- Private subnet for database (no direct route out)
- Security group on DB: allow only from web server
- NAT Gateway if DB needs outbound access (e.g. patches)

Cloud Network Services

- 1 From Virtualization to Cloud
- 2 Fundamentals of Cloud Computing
- 3 Cloud & Container Platforms
- 4 Cloud Network Architecture
- 5 Cloud Network Services**
- 6 The Cloud Trade-Off

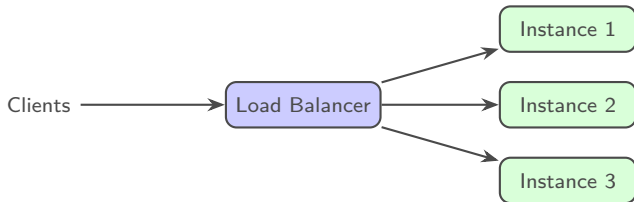
CloudBite Asks: “We Need HTTPS, Scaling, and Office Access”

- CloudBite’s app is live, but traffic is growing fast
- A single web server cannot handle the load; they need **traffic distribution**
- The API must be reachable via HTTPS, but the backends should stay hidden
- The CTO also wants to access the cloud network from the office without exposing it to the Internet

Three services solve these problems: load balancers, proxies, and VPNs.

Load Balancer

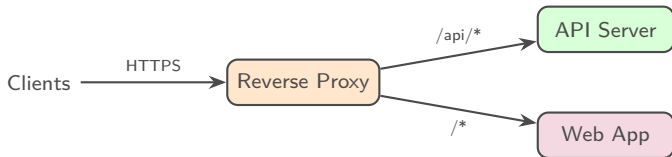
- Distributes incoming traffic across **multiple instances** (VMs or containers)
- Prevents any single instance from being overwhelmed
- If one instance fails, traffic is **redirected** to healthy ones



- **L4 (Transport Layer)** Load Balancer: routes based on IP and port (fast, simple)
- **L7 (Application Layer)** Load Balancer: routes based on HTTP headers, URL path, cookies (smarter)
- Cloud examples: AWS ALB/NLB, Azure Load Balancer, GCP Cloud Load Balancing

Reverse Proxy

- Sits **in front of** backend servers, receives all client requests
- Clients never communicate directly with the backend

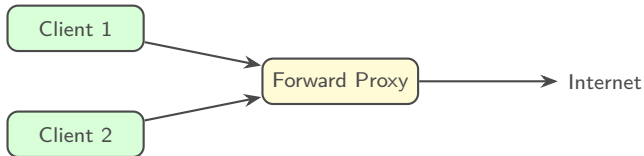


Key functions:

- **TLS (Transport Layer Security) termination:** handles HTTPS encryption, backends receive plain HTTP
- **URL routing:** /api/* → API server, /* → web app
- **Caching:** stores responses to reduce backend load
- Examples: NGINX, HAProxy, AWS CloudFront, Azure Front Door

Forward Proxy

- Sits **in front of clients**, controls their **outbound** traffic
- Clients send requests to the proxy, which forwards them to the Internet



Key functions:

- **Access control:** block certain websites or domains
- **Caching:** store frequently accessed content, reduce bandwidth
- **Anonymity:** external servers see the proxy's IP, not the client's
- **Logging:** monitor what employees or VMs access
- Examples: Squid, corporate firewalls with proxy mode

Reverse Proxy vs Forward Proxy

	Forward Proxy	Reverse Proxy
Protects	Clients (outbound)	Servers (inbound)
Position	In front of clients	In front of servers
Who knows?	Server sees proxy, not client	Client sees proxy, not server
Use case	Control outbound access	TLS termination, routing, caching

Think of it this way:

- **Forward proxy:** “I control what my users can access outside”
- **Reverse proxy:** “I control how outside users reach my servers”

VPN: Virtual Private Network

- A **VPN (Virtual Private Network)** creates a **secure, encrypted tunnel** over the public Internet
- Connects remote networks or users as if they were on the same private network



Two main types:

- **Site-to-site VPN:** connects two networks (e.g. office ↔ cloud VPC)
- **Client VPN:** individual user connects to a remote network (e.g. employee working from home)
- Cloud examples: AWS VPN, Azure VPN Gateway, GCP Cloud VPN

Discussion: Cloud Network Services

A company has a public frontend, a private API, and a database. Employees also need to access the cloud from the office. Which services would you use?

Think about:

- Load balancer in front of frontend (public)
- Reverse proxy to route `/api/*` to API server
- Database in private subnet, no public IP
- Site-to-site VPN for office access
- Security groups + ACLs at each tier

The Cloud Trade-Off

- 1 From Virtualization to Cloud
- 2 Fundamentals of Cloud Computing
- 3 Cloud & Container Platforms
- 4 Cloud Network Architecture
- 5 Cloud Network Services
- 6 The Cloud Trade-Off**

CloudBite, Five Years Later

- CloudBite grew to 2 million users. AWS bill: **\$40,000/month** and rising
- Egress fees alone: \$5,000/month (data leaving AWS to users)
- The team now has 30 engineers; they *could* run their own servers
- The CTO asks: “Should we leave the cloud?”

The cloud is not always the cheapest option. Let us explore why.

Case Study: Basecamp Leaves the Cloud

- **Basecamp / 37signals** (creators of Ruby on Rails) ran on AWS for 15 years
- In 2023, they **moved everything to their own hardware**: no new staff needed
- Result: **\$10 million saved over five years** (50%+ cost reduction)

Why it worked for them:

- **Stable, predictable workload**: no sudden traffic spikes
- **Large, experienced ops team**: already managing servers regardless
- **No need for global presence**: most users in the same region
- They built **Kamal**, an open-source deployment tool, to replace Kubernetes

Key insight

Basecamp's exit worked because their workload was **predictable**. For a startup with unpredictable traffic, the cloud's elasticity remains essential.

Vendor Lock-In: The Hidden Cost

- Cloud providers design services that are **easy to adopt, hard to leave**
- Proprietary services create **architectural dependencies**:
 - AWS Lambda, DynamoDB, SQS → no direct equivalent on Azure or GCP
 - Rewriting the application can cost more than years of cloud bills
- **Data egress fees** make leaving expensive:
 - AWS charges \$0.09/GB for outbound traffic
 - Moving 50 TB out of AWS $\approx$ \$4,500 in fees alone
 - Egress fees increased 20% in 2023; inter-Availability-Zone (AZ) fees doubled in 2025
- **68% of enterprises** exceeded their cloud budget due to unexpected data transfer costs (Flexera, 2024)

8

⁸Flexera, "State of the Cloud Report," 2024; AWS pricing documentation, 2025.

The Pricing Ratchet

- Cloud providers attract customers with **low initial pricing and free tiers**
- Once adoption deepens, prices increase:
 - AWS EC2 instance families: significant price rises in 2025
 - Azure: 5%+ on subscriptions, 10% on Premium SSDs (2025)
 - Google Workspace: 20%–34% increase (March 2025)
- This pattern appears across industries:
 - **Video streaming:** low subscription prices to gain users, then steady increases once the audience is locked in
 - **Cloud computing:** same dynamic, but switching costs are higher because your entire architecture depends on the provider

The pattern

Low prices → adoption → dependency → price increases → switching is too expensive.

So... Cloud or Not?

	Cloud (public)	On-premises / private
Best when	Unpredictable traffic, small team	Stable workload, large ops team
Cost model	OpEx (pay-as-you-go)	CapEx (buy hardware upfront)
Scaling	Instant, global	Weeks to months (buy + install)
Risk	Vendor lock-in, price hikes	Hardware failures, staffing
Control	Limited (provider's rules)	Full (your hardware, your rules)

- **Most real-world deployments are hybrid:** critical/stable workloads on-premises, burst/variable workloads in the cloud
- There is no universal answer; **every case is a trade-off**

Discussion: The Cloud Trade-Off

CloudBite (now 30 engineers, 2M users, stable traffic) pays \$40K/month on AWS. Should they leave the cloud, go hybrid, or stay? Why?

Think about:

- Is the workload predictable or bursty?
- Does the team have ops expertise?
- How much vendor-specific code exists (Lambda, DynamoDB)?
- What would migration cost in time and money?
- Could a hybrid approach capture the best of both?

Session Summary

- 1 From Virtualization to Cloud
- 2 Fundamentals of Cloud Computing
- 3 Cloud & Container Platforms
- 4 Cloud Network Architecture
- 5 Cloud Network Services
- 6 The Cloud Trade-Off

Key Takeaways: CloudBite's Journey

What we built:

1. Virtualization → cloud: technology vs **business model**
2. Five NIST characteristics define true cloud
3. **IaaS / PaaS / SaaS**: who manages what
4. Chose a **platform** (AWS, public cloud)
5. Created a **virtual network** with public and private subnets
6. Secured it with **security groups** + **ACLs**
7. Added **load balancer**, **reverse proxy**, **VPN**

What we learned:

8. The cloud is **not free**: costs grow with scale
9. **Vendor lock-in** makes leaving expensive
10. Prices increase once you depend on the provider
11. On-premises works for **stable, predictable** workloads
12. **Hybrid** is often the pragmatic answer
13. **Every case is a trade-off**; there is no universal solution

References

1. NIST SP 800-145, Mell & Grance, "The NIST Definition of Cloud Computing," 2011.
2. NIST SP 800-144, "Guidelines on Security and Privacy in Public Cloud Computing," 2011.
3. AWS, "Amazon VPC User Guide," docs.aws.amazon.com.
4. Microsoft Azure, "Azure Virtual Network Documentation," docs.microsoft.com.
5. Google Cloud, "VPC Documentation," cloud.google.com/vpc/docs.
6. CSA, "Security Guidance for Critical Areas of Focus in Cloud Computing v5," 2022.
7. Erl, Puttini & Mahmood, *Cloud Computing: Concepts, Technology & Architecture*, Prentice Hall, 2013.
8. Kurose & Ross, *Computer Networking: A Top-Down Approach*, 8th ed., Pearson, 2021.
9. AWS, "Elastic Load Balancing Documentation," docs.aws.amazon.com.
10. NGINX, Inc., "What Is a Reverse Proxy?," nginx.com/resources.
11. AWS, "AWS VPN Documentation," docs.aws.amazon.com.